

개발 회고

1. 개발 진행 방식

이번 프로젝트에서는 WakeBuddy 앱을 개발하면서 단순히 AI에게 바로 코드를 생성하도록 요청하는 방식이 아니라, **요구사항 분석 → 설계 → 구현 → 테스트 → 품질관리**의 개발 과정을 거쳐 코드를 작성하였다.

전체적인 개발 흐름은 소프트웨어 공학 강의록에서 다룬 개발 프로세스를 기준으로 진행하였고, 구현 과정에서는 XP의 특징인 **작은 단위의 기능 구현, 빠른 피드백, 반복적인 수정** 방식을 일부 적용하였다.

즉, 처음부터 완성된 코드를 한 번에 만드는 것이 아니라, 기능을 나누어 구현하고 실행 결과를 확인한 뒤 부족한 부분을 수정하는 방식으로 진행하였다.

2. 요구 분석

2.1 문제 정의

기존 알람 앱은 대부분 개인이 자신이 알람된 상황과 관리하는 방식이다. 그러나 친구, 가족, 집원처럼 서로의 일정을 함께야 하는 관계에서는 한 사람이 다른 사람에게 알람을 설정해 주거나, 상대방의 알람을 확인하고 수정할 수 있는 기능이 필요할 수 있다.

WakeBuddy는 이러한 문제를 해결하기 위해 연결된 사용자 간 알람 공유 기능을 제공한다.

2.2 도메인 분석

WakeBuddy의 주요 도메인은 다음과 같다.

도메인 개념	설명
사용자	앱을 사용하는 주체
친구	사용자와 연결된 다른 사용자
알람	특정 시간에 울리도록 설정된 정보
친구 알람	연결된 친구에게 생성한 알람
알람 관리	알람을 조회, 수정, 삭제할 수 있는 행위
알림/알람 실행	설정된 시간에 사용자에게 알람을 전달하는 기능

2.3 기능 요구와 비기능 요구 구분

WakeBuddy의 요구사항은 크게 기능 요구와 비기능 요구로 구분된다.

분류	설명	예시
기능 요구	시스템이 의도적으로 제공해야 하는 기능	알람 생성, 친구 추가, 친구 알람 수정
비기능 요구	시스템이 갖추어야 할 품질 조건	사용성, 유지보수성, 동시성 지원성

WakeBuddy 설계서

1. 아키텍처 설계

1.1 설계 개요

WakeBuddy는 React Native, Expo, TypeScript를 기반으로 Android와 iOS에서 동작하는 모바일 앱으로 설계한다.

1.2 아키텍처 구조

WakeBuddy는 다음과 같은 계층 구조로 설계한다.

```
[시뮬리]
├──
├── [React Native / Expo App]
├──
├── [Screen Components]
├──   ├── LoginScreen
├──   ├── LogoutScreen
├──   ├── HomeScreen
├──   ├── MyAlarmScreen
├──   ├── AlarmManagementScreen
├──   ├── FriendListScreen
├──   └── FriendAddScreen
├──
├── [Service Modules]
├──   ├── AuthService
├──   ├── AlarmService
├──   ├── FriendService
├──   ├── NotificationService
├──   └── NotificationService
├──
├── [External Infrastructure]
├──   ├── Firebase Authentication
├──   ├── Cloud Firestore
├──   └── Expo Notifications
```

전체 구현 기능

WakeBuddy에서 최종적으로 구현할 기능은 다음과 같다.

구분	기능
사용자 인증	회원가입, 로그인, 로그아웃
내 알람 관리	내 알람 조회, 생성, 수정, 삭제, 활성화/비활성화
친구 관리	친구 목록 조회, 친구 추가, 친구 삭제
친구 알람 관리	친구 알람 조회, 생성, 수정, 삭제
데이터 저장	사용자, 알람, 친구 관계 데이터 저장
알림 기능	알람 관리 요청, 알람 백역, 알람 취소
품질 확인	입력값 검증, 오류 메시지 표시, 기능별 동적 테스트

오늘 구현할 기능

오늘은 MVP 구현을 목표로 한다. MVP는 실제 서버나 알림 기능까지 완벽하게 만들려기보다, 앱 화면과 핵심 기능 흐름이 동작하는 상태를 의미한다.

구분	오늘 할 일
개발 환경	Expo 프로젝트 생성 및 실행 확인
프로젝트 구조	src/components, services, models, firebase, constants, utils 폴더 생성
화면 구현	Home 화면, Login 화면, MyAlarm 화면, AlarmForm 화면, FriendList 화면 구현
알람 기능	임시 데이터 기반 내 알람 조회, 생성, 수정, 삭제, 활성화/비활성화
친구 기능	임시 데이터 기반 친구 목록 조회, 친구 추가, 친구 삭제
데이터 모델	User, Alarm, Friendship 타입 정의
GitHub 관리	기능 단위로 커밋하여 구현 과정 기록

오늘 구현에서는 Firebase와 실제 알림 기능을 바로 연결하지 않고, 임시 데이터를 사용하여 화면과 기능 흐름을 먼저 만든다.

2. FriendRequestService 단위 테스트 보고서

1. 테스트 개요

항목	내용
테스트 대상	FriendRequestService.ts
테스트 단계	단위 테스트 (Unit Test)
테스트 기법	블랙박스 테스트 — 동등분할 + 경계값 분석 + 상태 기반 테스트
테스트 도구	Jest + ts-jest
테스트 일자	2026-05-25
결과	18/18 PASS

2. 테스트 케이스 명세서

sendFriendRequest 동등분할

테스트 대상	테스트 조건	테스트 데이터	예상 결과	실제 결과
sendFriendRequest	정상 입력 (Valid)	senderId, receiverEmail 정상	친구 요청 생성 성공	PASS
sendFriendRequest	senderId 없음 (Invalid)	senderId: ""	에러: "로그인 정보가 없습니다."	PASS
sendFriendRequest	receiverEmail 없음 (Invalid)	receiverEmail: ""	에러: "친구 이메일을 입력해주세요."	PASS
sendFriendRequest	존재하지 않는 이메일 (Invalid)	없는 이메일	에러: "해당 이메일의 사용자를 찾을 수 없습니다."	PASS

7.1 입력값 검증

입력값 검증 로직은 AuthValidationUtils.ts 로 분리하였다.

```
export function validateLoginInput(email: string, password: string): void {
  if (!email.trim()) {
    throw new Error("이메일을 입력해주세요.");
  }

  if (!password.trim()) {
    throw new Error("비밀번호를 입력해주세요.");
  }
}
```

```
export function validateSignUpInput(
  email: string,
  password: string,
  displayName: string,
): void {
  if (!email.trim()) {
    throw new Error("이메일을 입력해주세요.");
  }

  if (!password.trim()) {
    throw new Error("비밀번호를 입력해주세요.");
  }

  if (!displayName.trim()) {
    throw new Error("이름을 입력해주세요.");
  }
}
```

2. 요구사항 분석 단계 회고

요구사항 분석 단계에서는 먼저 WakeBuddy 앱이 제공해야 할 핵심 기능을 정리하였다.

주요 요구사항은 다음과 같았다.

구분	내용
알람 관리	사용자는 본인의 알람을 생성, 조회, 수정, 삭제할 수 있다.
친구 기능	사용자는 다른 사용자에게 친구 요청을 보낼 수 있다.
친구 요청 처리	친구 요청을 받은 사용자는 요청을 수락하거나 거절할 수 있다.
공유 알람 관리	친구 관계가 성립된 사용자끼리는 서로의 알람을 관리할 수 있다.
권한 검증	친구 관계가 아닌 사용자는 상대방의 알람을 관리할 수 없다.
알림 예약	설정된 시간에 알림이 울리도록 예약할 수 있다.
입력값 검증	로그인, 회원가입, 알람 생성 시 잘못된 입력을 방지해야 한다.

이 과정을 통해 단순히 “공유 알람 앱을 만들어 달라”는 수준이 아니라, **어떤 기능이 필요하고 어떤 조건이 지켜져야 하는지**를 구체적으로 정리할 수 있었다.

어려웠던 점

처음에는 기능을 단순하게 “알람 기능”, “친구 기능”, “로그인 기능” 정도로만 생각하였다. 하지만 요구사항을 정리하다 보니 각 기능 안에도 세부 조건이 많다는 것을 알게 되었다.

예를 들어 친구 기능의 경우 단순히 친구 목록만 있으면 되는 것이 아니라, 친구 요청을 보내는 과정, 요청을 수락하거나 거절하는 과정, 친구 관계가 된 이후 권한을 확인하는 과정이 필요했다. 또한 공유 알람 기능에서도 “누가 누구의 알람을 수정할 수 있는가”와 같은 조건을 명확히 해야 했다.

해결 방안

기능을 큰 단위로만 보지 않고, 사용자의 행동 흐름에 따라 세부 기능을 나누었다.

예를 들어 친구 기능은 다음과 같이 나누었다.

친구 요청 보내기 → 친구 요청 조회 → 요청 수락/거절 → 친구 관계 생성 → 친구 알람 관리 권한 확인

이렇게 나누면서 요구사항이 더 명확해졌고, 이후 설계 단계에서 어떤 서비스가 필요한지도 정리할 수 있었다.

3. 설계 단계 회고

설계 단계에서는 요구사항을 바탕으로 기능별 책임을 분리하는 데 집중하였다.

기능을 화면 코드에 모두 넣는 대신, 서비스와 유틸리티를 나누어 각 파일이 하나의 역할을 담당하도록 구성하였다.

파일	역할
<code>AlarmService</code>	알람 생성, 조회, 수정, 삭제 관리
<code>AuthService</code>	로그인, 회원가입 등 인증 관련 기능 처리
<code>FriendRequestService</code>	친구 요청 생성, 수락, 거절 처리
<code>FriendService</code>	친구 관계 조회 및 관리
<code>NotificationService</code>	알림 예약 및 취소 처리
<code>PermissionService</code>	사용자 권한 검증
<code>AlarmDateUtils</code>	알람 날짜 및 시간 계산
<code>AuthValidationUtils</code>	로그인/회원가입 입력값 검증
<code>alarmSchedule</code>	알람 반복 일정 관련 로직 처리

이를 통해 기능별 책임이 명확해졌고, 특정 기능을 수정할 때 어느 파일을 확인해야 하는지 파악하기 쉬워졌다.

어려웠던 점

설계 단계에서 가장 어려웠던 점은 **기능을 어디까지 나누어야 하는지 결정하는 것이었다**. 너무 많이 나누면 파일 수가 많아져 복잡해지고, 너무 적게 나누면 하나의 파일에 여러 책임이 몰릴 수 있었다.

예를 들어 친구 기능의 경우 `FriendService` 하나에 친구 요청까지 모두 넣을 수도 있었지만, 그렇게 하면 친구 요청과 친구 관계 관리가 섞이게 된다. 알람 기능도 알람 생성과 알림 예약을 하나로 처리할 수도 있었지만, 그러면 알람 데이터 관리와 실제 알림 예약 책임이 섞일 수 있었다.

해결 방안

기준을 “변경 이유가 다르면 파일을 분리한다”로 잡았다.

예를 들어 친구 요청 로직은 요청 상태가 변경될 때 수정될 가능성이 크고, 친구 관계 로직은 실제 친구 목록이나 친구 여부 판단이 변경될 때 수정될 가능성이 크다. 따라서

`FriendRequestService` 와 `FriendService` 를 분리하였다.

또한 권한 검증은 여러 기능에서 공통으로 사용될 수 있기 때문에 `PermissionService` 로 분리하였다. 이로 인해 알람 생성, 수정, 삭제 기능에서 권한 판단을 일관되게 사용할 수 있었다.

4. 구현 단계 회고

구현 단계에서는 전체 기능을 한 번에 완성하기보다 작은 단위로 나누어 진행하였다.

먼저 기본 알람 관리 기능을 구현하고, 이후 알림 예약 기능을 연결하였다. 그다음 친구 요청, 친구 관계, 권한 검증 기능을 차례대로 추가하였다. 기능을 구현한 뒤에는 바로 실행하거나 테스트하면서 문제가 있는 부분을 수정하였다.

이 방식은 XP의 작은 단위 개발과 빠른 피드백 방식과 유사하게 진행되었다.

구현 과정 예시

1. 알람 데이터 구조 정의
2. 알람 생성/조회/수정/삭제 기능 구현
3. 알림 예약 기능 연결
4. 친구 요청 기능 구현
5. 친구 관계 관리 기능 구현
6. 친구 여부에 따른 권한 검증 추가
7. 입력값 검증 및 예외 처리 보완
8. 테스트 코드 작성 및 수정

어려웠던 점

구현 과정에서는 기능 간 연결이 어려웠다.

알람 기능만 보면 단순히 알람을 생성하면 되지만, 실제로는 알람 생성 시 입력값 검증, 알림 예약, 권한 검증이 함께 필요했다.

또한 친구 알람 기능의 경우 현재 사용자가 본인의 알람을 다루는지, 친구의 알람을 다루는지에 따라 권한 판단이 달라져야 했다. 이처럼 기능들이 서로 연결되면서 단순 구현보다 복잡해졌다.

해결 방안

한 기능 안에서 모든 처리를 하지 않고, 역할에 따라 로직을 나누었다.

예를 들어 알람 생성 과정은 다음처럼 분리해서 생각하였다.

입력값 확인 → 권한 확인 → 알람 데이터 생성 → 알림 예약 → 저장 또는 결과 반영

이렇게 흐름을 나누니 어느 부분에서 문제가 발생했는지 확인하기 쉬웠다. 또한 반복적으로 사용되는 날짜 계산이나 입력값 검증은 유틸리티 함수로 분리하여 중복을 줄였다.

5. 테스트 단계 회고

테스트 단계에서는 핵심 로직을 중심으로 테스트 코드를 작성하였다.

테스트 대상은 화면 전체보다는 요구사항을 만족하는 데 중요한 내부 로직을 중심으로 선정하였다.

테스트 대상	테스트 목적
<code>AlarmDateUtils.test.ts</code>	알람 시간 계산이 올바르게 되는지 확인
<code>AuthValidationUtils.test.ts</code>	로그인/회원가입 입력값 검증이 올바른지 확인
<code>PermissionService.test.ts</code>	친구 관계에 따른 권한 판단이 올바른지 확인

테스트를 통해 코드가 단순히 실행되는지만 확인하는 것이 아니라, 요구사항에서 정한 조건을 실제로 만족하는지 검증할 수 있었다.

어려웠던 점

처음에는 어떤 부분을 테스트해야 할지 정하기 어려웠다. 화면을 직접 눌러보는 수동 테스트는 비교적 쉽게 할 수 있었지만, 자동 테스트 코드는 입력값과 예상 결과를 명확히 정해야 했다.

특히 권한 검증 테스트는 조건이 여러 개라 어려웠다. 본인 알람인지, 친구 알람인지, 친구 관계가 있는지, 없는지에 따라 결과가 달라졌기 때문이다.

해결 방안

테스트 대상을 모든 기능으로 넓히기보다, 오류가 발생하면 치명적인 핵심 로직부터 선택하였다.

우선순위는 다음과 같이 잡았다.

- 1순위: 권한 검증
- 2순위: 입력값 검증
- 3순위: 알람 시간 계산

이 기준에 따라 테스트 코드를 작성하였고, 각 테스트는 “입력 → 실행 → 예상 결과 확인” 구조로 정리하였다. 이를 통해 기능 수정 후에도 기존 로직이 유지되는지 확인할 수 있었다.

6. 품질관리 단계 회고

품질관리 단계에서는 코드가 단순히 동작하는지를 넘어서, 유지보수성과 안정성을 확인하는데 집중하였다.

주요 확인 기준은 다음과 같았다.

품질관리 기준	확인 내용
책임 분리	한 파일이 너무 많은 기능을 담당하지 않는지 확인
입력값 검증	잘못된 입력을 사전에 막을 수 있는지 확인
권한 검증	허용되지 않은 사용자의 접근을 막을 수 있는지 확인
예외 처리	비정상 상황에서 앱이 중단되지 않도록 처리했는지 확인
테스트 가능성	핵심 로직이 테스트 가능한 형태로 분리되어 있는지 확인
코드 이해도	함수와 파일의 역할이 명확한지 확인

어려웠던 점

품질관리는 명확한 정답이 있는 작업이 아니라 판단이 필요한 작업이었다.

예를 들어 코드가 동작하더라도, 그 코드가 좋은 구조인지, 나중에 수정하기 쉬운지, 테스트하기 쉬운지는 별도로 검토해야 했다.

또한 기능을 추가하다 보면 기존에 분리해둔 구조가 부족해지는 경우도 있었다. 처음에는 적절해 보였던 파일 구조가 기능이 늘어나면서 다시 수정이 필요해지는 경우가 있었다.

해결 방안

품질관리 기준을 코드 구조, 테스트 가능성, 중복 여부 중심으로 잡았다.

특히 반복되는 로직이 있으면 유틸리티로 분리하고, 여러 기능에서 공통으로 필요한 판단 로직은 서비스로 분리하였다.

또한 테스트 코드가 작성 가능한 구조인지 확인하였다. 테스트하기 어려운 코드는 화면이나 외부 기능에 너무 강하게 의존하고 있을 가능성이 있기 때문에, 핵심 로직을 별도 함수로 분리하려고 하였다.

7. AI 직접 생성 코드와 비교하며 느낀 점

AI에게 바로 코드를 생성하도록 요청한 방식은 빠르게 결과물을 얻을 수 있다는 장점이 있었다. 화면 구성과 기본 기능이 빠르게 만들어졌기 때문에 초기 아이디어 확인이나 프로토타입 제작에는 효과적이었다.

하지만 두 코드를 직접 비교해보면서 처음에는 보이지 않던 차이가 눈에 들어오기 시작했다. AI가 생성한 코드도 기능 자체는 동작했기 때문에 처음에는 "뭐가 문제인가"라는 생각이 들었다.

그러나 직접 `PermissionService`를 설계하고 `FriendRequestService`와 `FriendService`를 분리해보고 나니, 이런 구조가 없을 때 어떤 문제가 생기는지 실감할 수 있었다. 단순히 결과물을 비교하는 것이 아니라, 왜 이런 차이가 생기는지 이해하게 된 것이다.

또한 처음에는 설계 단계에 시간을 쓰는 것이 아깝게 느껴졌다. 하지만 기능을 추가하거나 수정할 때 어느 파일을 고쳐야 할지 바로 파악할 수 있었고, 결과적으로 오히려 시간이 절약됐다. 테스트 코드를 작성하는 과정에서도 예상치 못한 효과가 있었다. 테스트 케이스를 짜다 보니 "이 조건을 빠뜨렸네" 하고 요구사항 누락을 발견하는 경우가 있었고, 이는 수동 테스트만으로는 잡기 어려운 부분이었다.

이번 비교를 통해 AI 활용 방식에 대한 생각도 달라졌다. AI는 코드를 빠르게 생성하는 도구로만 쓰기보다, 요구사항 정리나 테스트 케이스 아이디어를 얻는 용도로 활용하는 것이 더 효과적이라는 것을 느꼈다. 코드 생성 자체보다 개발 과정을 보조하는 역할로 쓸 때 더 좋은 결과가 나온다는 점을 직접 경험할 수 있었다.

구분	AI 직접 생성 코드	프로세스 적용 코드
개발 속도	빠름	상대적으로 느림
구조화	단순함	기능별 책임 분리
요구사항 반영	기본 기능 중심	세부 조건까지 반영
테스트	부족함	핵심 로직 테스트 포함
품질관리	수동 확인 중심	검증 기준을 두고 점검
유지보수성	기능 증가 시 어려움 가능	수정 범위 파악이 쉬움

8. 전체적으로 어려웠던 점

이번 개발 과정에서 어려웠던 점은 크게 네 가지였다.

첫째, 요구사항을 구체화하는 과정이 어려웠다.

처음에는 필요한 기능이 단순해 보였지만, 실제로는 친구 요청, 권한 검증, 알람 반복, 입력값 검증처럼 세부 조건이 많았다.

둘째, 설계 구조를 나누는 기준을 정하기 어려웠다.

기능을 너무 적게 나누면 코드가 복잡해지고, 너무 많이 나누면 파일 수가 많아져 이해가 어려워질 수 있었다.

셋째, 테스트 코드를 작성하는 과정이 어려웠다.

특히 어떤 값을 입력하고 어떤 결과를 기대해야 하는지 명확히 정해야 했기 때문에 수동 테스트보다 더 많은 고민이 필요했다.

넷째, 기능 구현과 품질관리 사이의 균형을 맞추기 어려웠다.

빠르게 기능을 완성하고 싶지만, 동시에 코드 구조와 테스트도 신경 써야 했기 때문에 시간이 더 필요했다.

9. 해결 방안 및 개선 방향

어려움을 해결하기 위해 기능을 작은 단위로 나누고, 각 기능별 책임을 명확히 하려고 하였다. 또한 복잡한 로직은 서비스와 유틸리티로 분리하여 화면 코드에 모든 기능이 몰리지 않도록 하였다.

테스트의 경우 모든 기능을 한 번에 테스트하려고 하지 않고, 권한 검증, 입력값 검증, 알람 시간 계산처럼 오류가 발생하면 중요한 문제가 되는 핵심 로직부터 테스트하였다.

앞으로 개선할 점은 다음과 같다.

개선 방향	내용
테스트 범위 확대	서비스 로직뿐만 아니라 화면 흐름과 통합 테스트도 추가
코드 리뷰 강화	팀원 간 코드 검토를 통해 오류와 중복을 조기에 발견
요구사항 문서화 개선	기능별 조건과 예외 상황을 더 명확하게 기록
리팩토링 주기 확보	기능 구현 후 구조 개선 시간을 별도로 확보
AI 활용 방식 개선	코드 생성보다는 요구사항 정리, 테스트 케이스 작성, 리팩토링 보조로 활용

10. 최종 회고

이번 프로젝트를 통해 단순히 동작하는 코드를 만드는 것과, 요구사항 분석과 설계, 테스트, 품질관리를 거쳐 코드를 만드는 것은 차이가 있다는 점을 확인하였다.

AI를 직접 활용하면 빠르게 앱의 형태를 만들 수 있지만, 요구사항의 세부 조건을 반영하고 안정적인 구조를 만들기 위해서는 별도의 분석과 설계 과정이 필요했다. 특히 친구 관계에 따른 권한 검증, 입력값 검증, 알람 시간 계산처럼 앱의 핵심이 되는 로직은 테스트와 품질관리를 통해 확인하는 과정이 중요했다.

또한 XP 방식처럼 작은 단위로 기능을 구현하고, 피드백을 통해 반복적으로 수정하는 과정은 개발 중 발생하는 문제를 빠르게 발견하는 데 도움이 되었다. 다만 이번 프로젝트의 핵심은 XP 자체라기보다는, 소프트웨어 공학 개발 프로세스를 실제 코드 작성에 적용해보았다는 점에 있다.

결과적으로 이번 개발을 통해 **빠른 구현도 중요하지만, 요구사항을 명확히 하고 설계와 테스트를 거쳐 품질을 관리하는 과정이 코드의 유지보수성과 안정성에 큰 영향을 준다는 것**을 알 수 있었다. 앞으로는 AI를 단순히 코드를 대신 작성하는 도구로만 사용하기보다, 요구사항 정리, 테스트 케이스 작성, 코드 개선 방향 제안 등 개발 과정을 보조하는 도구로 활용하는 것이 더 효과적일 것으로 판단된다.